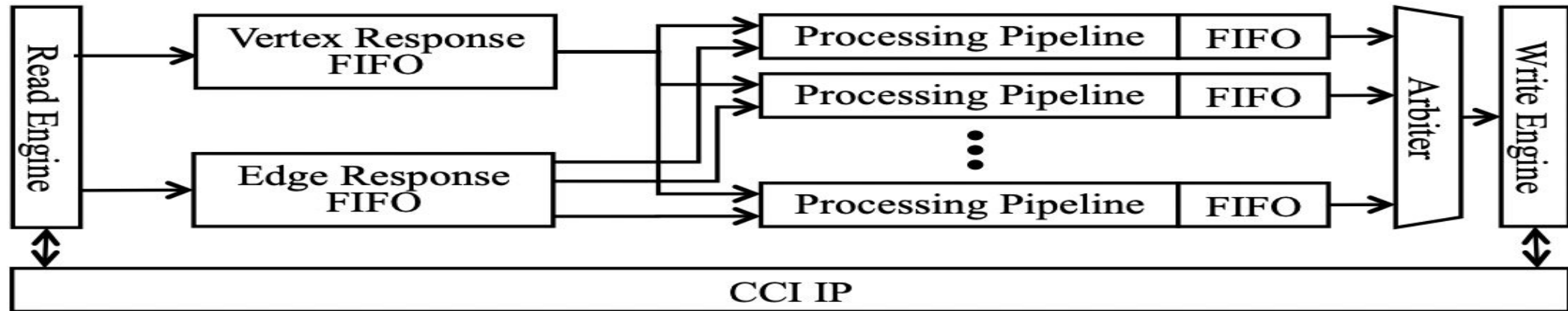


Problem Statement

The paper aims to implement and evaluate a work stealing scheduler for graph processing on heterogeneous CP+FPGA systems. FPGA stands for Field Programmable Gate Arrays. FPGAs are hardware accelerators and are prevalent in computing from personal to cloud computing. They can process parallel computations with high throughput.



Picture credits: <https://dl.acm.org/doi/pdf/10.1145/3404397.3404433>

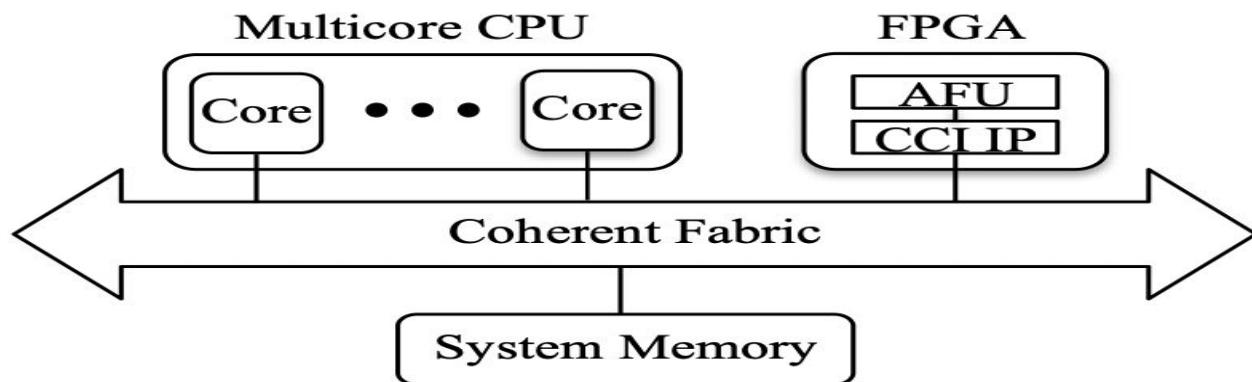
The paper compares performance with static scheduling, chunk scheduling and oracle chunk scheduling. The goal is to scale throughput with increasing threads.

Motivation

1.Graph processing is key in machine learning,social networks,bioinformatics.High throughput algorithms are required for graph processing

Name	Vertices	Edges	Description
Twitter	62M	1,468M	Twitter followers
LiveJournal	4.8M	69M	Friendship relations
Orkut	3M	234M	Social Connections
StackOverflow	2.6M	36M	Questions & Answers
Skitter	1.7M	22M	2005 Internet topology
Pokec	1.6M	31M	Social Connections
Higgs	460k	15M	Twitter subset

2.Accelerators(GPU,DSP,FPGA etc.) allow concurrent graph processing but incur overhead of copying data to and fro from the accelerator.FPGA can be coupled with CPU resulting in a shared memory system coherent with the caches of CPU



Contribution

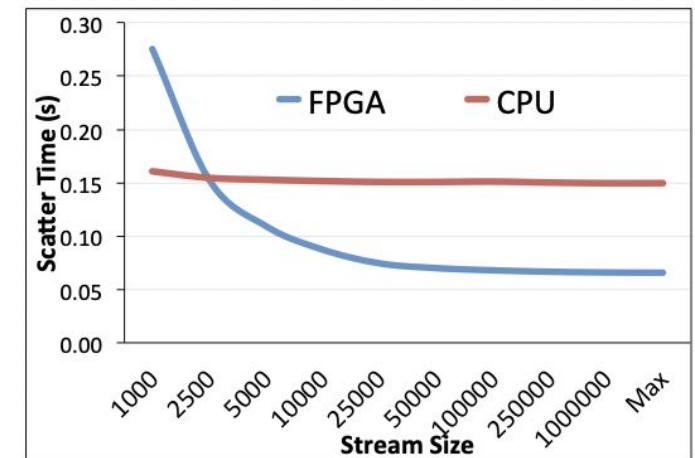
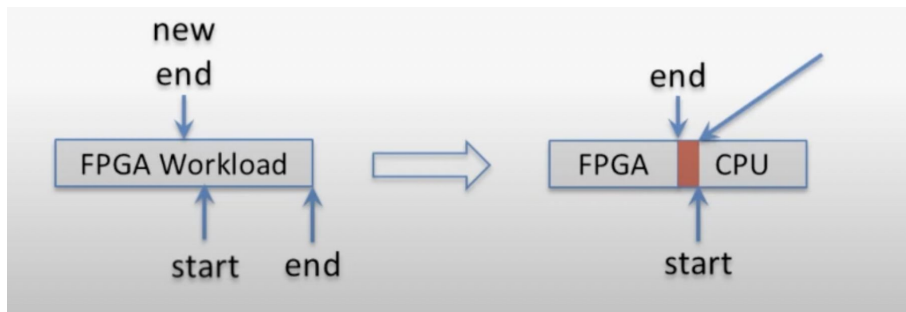
1. Provide the first work stealing algorithm for graph processing for heterogeneous tightly coupled CPU+FPGA systems.
2. Provide an evaluation that effectively demonstrates the efficiency of heterogeneous work stealing scheduler on CPU+FPGA systems.



Implementation

Challenges:

1. Workload is a **contiguous stream** in the vertex buffer. Work stealing from deque will be inefficient.
2. Synchronisation results in paying additional cost of synchronisation. Polling might result in a worker being idle. Thief interrupt victim might lead to overhead.
3. Threads spawning other threads while performing computations.
4. Stealing when number of vertices remain very small will lead to thrashing.
5. Performance of AFU is proportional to length of stream assigned to it.
6. Stealing from AFU might lead to reinitialisation
7. How to steal from AFU?
8. AFU maintains cache granularity. What if vertex stream does not fit into cache line(s)?
9. Stale steals from AFU.



Implementation

Primitives:

1. Graphs are processed in a bulk synchronous manner. Fixed number of threads are initialised, work is divided to them. Mitigates the cost of thread creation and management
2. AFU is assigned a long enough vertex stream
3. CPU threads steal upper half (experimentally determined) of AFU stream to avoid reinitialisation
4. AFU requests memory for a cache line worth of its workload every cycle (this is by design)
5. Once a vertex is being processed by AFU, it cannot be stolen
6. Stealing disabled when less number of vertices remain (80).
7. Value of S and E (start and end vertex) are stored in CSR of AFU. They can be read by software via shared memory buffer



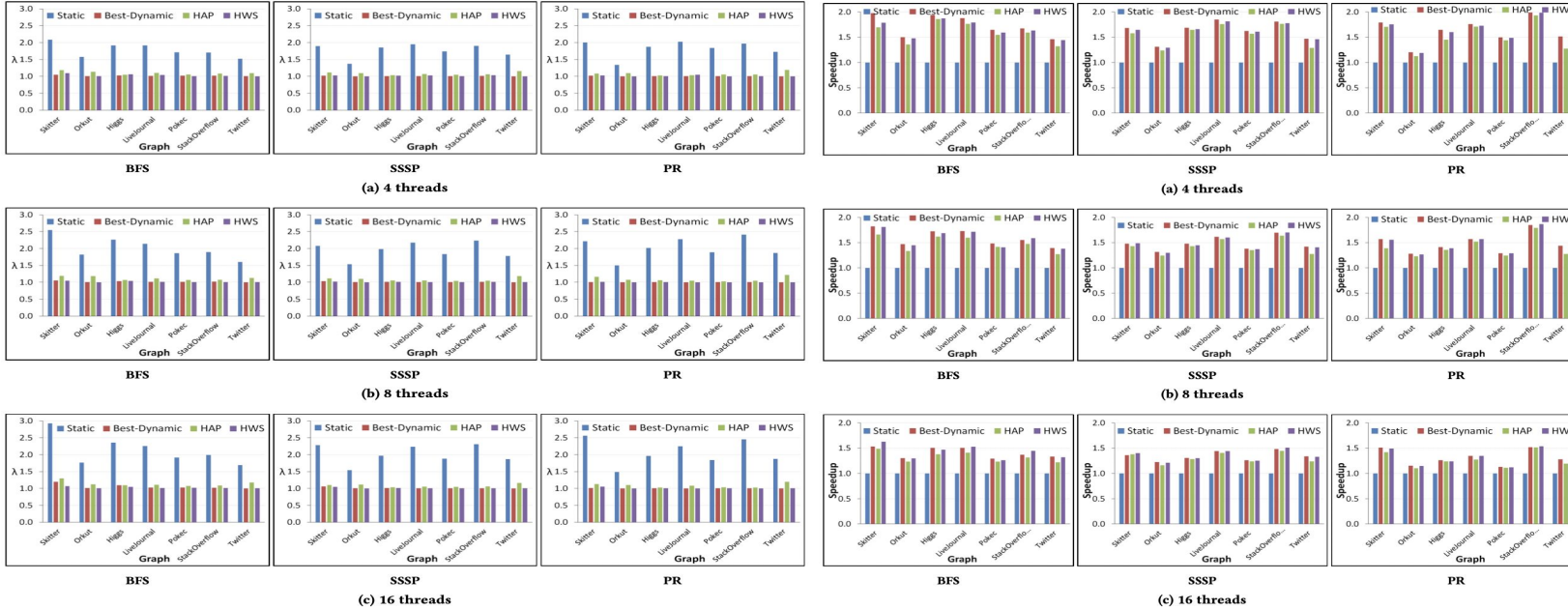
Implementation

```
1: Thread  $T[i](S, E, sync)$ 
2: while true do
3:   if  $S < E$  then
4:     Process:
5:     for each  $vtx$  from  $S$  to  $E$  do
6:       Scatter( $vtx$ )
7:        $S = S + 1$ 
8:   else ▷  $S == E$ 
9:     Steal:
10:     $k = \text{id of randomly chosen thread}$  ▷  $k \neq i$ 
11:    if compare_and_swap( $T[k].sync, false, true$ ) then
12:       $T[i].S = ((T[k].S + P) + T[k].E) / 2$  ▷ Steal half
13:       $T[i].E = T[k].E$ 
14:       $T[k].E = T[i].S$ 
```

Picture Credits: <https://dl.acm.org/doi/pdf/10.1145/3404397.3404433>

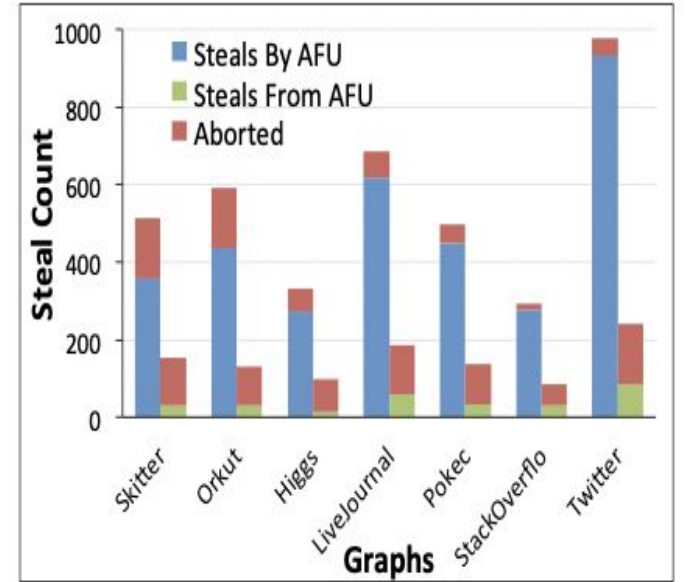
1. CAS to ensure one thief at victim k at a time
2. P (determined via AFU progress) to avoid stale steals from CPU.
3. Non synchronised steals from cache.

Evaluation Summary

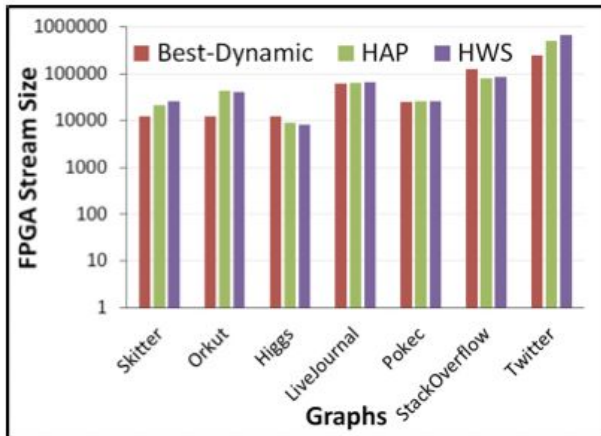


Value of λ for 4,8,16 threads

Normalized speedup for 4,8,16 threads



Number of steals



Average AFU stream size